

Staff DevOps Engineer – Technical Assessment

Company: CleverTap | **Role:** Staff DevOps Engineer

Time Expectation: 4–6 hours | **Submission Format:** GitHub repo + written document

Context

CleverTap processes **40+ billion events daily** across **4+ billion devices**. The platform is a multi-tenant SaaS product where customer campaigns trigger sudden, unpredictable traffic spikes — sometimes 10–50x baseline within minutes.

You are joining as a Staff DevOps Engineer. On Day 30, you are asked to **own the infrastructure modernization initiative** end-to-end. The current state is as follows:

Current State (what you inherit):

- Workloads running on EKS across two AWS regions (us-east-1, ap-south-1), but clusters are not standardized — each was provisioned differently over time.
- CDK is partially adopted; roughly 40% of infrastructure is still click-ops.
- CI/CD pipelines exist (GitHub Actions) but have no environment promotion gates, no drift detection, and secrets are hardcoded in some pipeline YAMLs.
- Observability is fragmented: CloudWatch for infra metrics, a self-hosted Prometheus/Grafana stack for app metrics, and no unified alerting strategy.
- The SRE team has flagged recurring issues: noisy alerts (>200 alerts/day, majority non-actionable), two P0 incidents in the last quarter caused by config drift, and no formal runbooks.
- Monthly AWS bill is \$420K. Finance has asked for a 25–30% reduction without compromising SLA.

Section 1: Infrastructure Architecture & IaC (35 points)

Problem

You need to standardize and harden the multi-region EKS infrastructure so it is reproducible, auditable, and secure.

Task

1a. Terraform/Cloudformation/CDK Module Design (implementation required)

Write production-grade code for the following. Your code will be evaluated on structure, reusability, and security — not just functionality.

- A reusable **EKS cluster module** that supports:
 - Private API server endpoint
 - IRSA (IAM Roles for Service Accounts) for workload identity
 - Node groups with mixed instance types (On-Demand + Spot) and a clear eviction strategy
 - Cluster add-ons managed via Terraform (e.g., VPC CNI, CoreDNS, kube-proxy)
- A **multi-region VPC module** that can be instantiated identically across regions with:
 - Public, private, and intra (database) subnets
 - VPC peering or AWS Transit Gateway between regions — justify your choice
 - VPC Flow Logs enabled and shipped to S3 with lifecycle policies

1b. State & Drift Management

- Describe (no code needed) how you would structure Terraform state across:
 - Multiple AWS accounts (dev, staging, prod)
 - Multiple regions
 - Multiple teams contributing to the same codebase
- What tooling would you introduce to detect and alert on infrastructure drift? How would you handle drift remediation?

1c. Design Question

The platform is expanding into the EU. Data residency laws require that EU customer data never leave EU-West-1. You must still maintain a single control plane for deployments.

How do you architect this? Describe your approach to cluster federation or isolation, IAM boundary enforcement, and how your CI/CD pipeline enforces the data residency constraint.

Section 2: Reliability, Observability & Incident Response (30 points)

Problem

The current observability setup is noisy and reactive. You need to build a system that enables fast detection, diagnosis, and resolution of production incidents.

Task

2a. Observability Stack Design

Design a unified observability architecture covering the four pillars: **metrics, logs, traces, and events**. Your design should:

- Specify tooling choices for each pillar (you may use AWS-native, OSS, or commercial tools — justify each)
- Define the data flow: collection → aggregation → storage → alerting
- Address **cardinality management** — a common scaling problem at 30B events/day
- Describe how you would implement **SLO-based alerting** (using error budget burn rate) instead of threshold-based alerting, and why this reduces alert noise

2b. Runbook & Incident Response (implementation required)

Write a **structured runbook** for the following scenario:

Alert fires: KubePodCrashLooping for a critical event-ingestion service in production. The service processes inbound campaign events and feeds a downstream Kafka topic. An on-call engineer with 6 months of experience is first responder.

Your runbook should cover:

- Initial triage steps (what to check first, in order)
- Decision tree: rollback vs. hotfix vs. scale-out
- Escalation criteria and communication template (internal + customer-facing)
- Post-incident artifact: what should the PIR (Post-Incident Review) document capture?

2c. Design Question

You discover that 60% of alerts are auto-resolving within 5 minutes with no human action. The on-call team is experiencing fatigue.

What is your systematic approach to reducing alert noise? Walk through how you would audit, classify, and remediate the alert backlog. What metrics would you use to measure the health of your alerting system going forward?

Section 3: Developer Platform, CI/CD & Engineering Velocity (20 points)

Problem

Deployment pipelines are inconsistent across teams. Some services deploy 10x/day; others deploy once a week due to fear of breakage. Your goal is to improve deployment confidence and velocity across the org.

Task

3a. CI/CD Pipeline Design (implementation required)

Design and partially implement a **GitHub Actions pipeline** for a containerized microservice deployed on EKS. The pipeline must include:

- **PR stage:** lint, unit tests, container image build, SAST scan (e.g., Trivy or Semgrep), and image push to ECR with commit-SHA tags
- **Staging promotion:** Helm-based deployment to staging, automated smoke tests, and a manual approval gate
- **Production promotion:** Canary deployment (10% → 50% → 100%) using Argo Rollouts or Flagger — describe the rollout strategy and automated rollback triggers
- **Secret management:** No secrets in YAML. Describe how secrets are injected at runtime (e.g., AWS Secrets Manager + External Secrets Operator)

Provide the GitHub Actions YAML for the PR and staging stages. The production stage can be described in prose with relevant config snippets.

3b. Design Question

A product engineering team wants to self-serve infrastructure provisioning — they want to spin up isolated environments (databases, queues, caches) for feature testing without waiting for the DevOps team.

Design an Internal Developer Platform (IDP) capability that allows this safely.

How do you balance speed-of-provisioning with cost governance, security guardrails, and cleanup automation?

Section 4: Cost Engineering (15 points)

Problem

The current \$420K/month AWS bill needs to be reduced by 25–30% (\$105–126K savings) without impacting SLA commitments.

Task

4a. Cost Reduction Plan

Given the workload profile (EKS on EC2, RDS, ElastiCache, S3, heavy data transfer between regions), propose a **structured 90-day cost reduction plan** with:

- Quick wins (Week 1–2): what can be actioned immediately?
- Medium-term changes (Month 1–2): right-sizing, commitment strategy (Savings Plans vs Reserved Instances — when to use which?)
- Architectural changes (Month 2–3): any workload migrations or redesigns that unlock structural savings

For each initiative, provide:

- Estimated savings (% or \$)
- Implementation effort (Low / Medium / High)
- Risk to reliability (Low / Medium / High)

4b. FinOps Process Design

As a Staff Engineer, you are not just expected to cut costs once — you are expected to **institutionalize cost visibility**.

Describe the tagging strategy, showback/chargeback model, and alerting thresholds you would put in place so that engineering teams own their own cloud spend.
